

Proof Buddy — Directory Map

This document explains every significant directory and file in the repository. It is intended to orient someone completely new to the project — human developer or AI assistant — so they can quickly locate any piece of logic.

Top-level Files

File	Purpose
<code>README.md</code>	Entry-point documentation for humans. Points to detailed setup guides in documentation/ .
<code>docker-compose.yml</code>	Defines three Docker services: <code>django_server</code> , <code>nginx</code> , <code>client</code> . Wires together the full production stack.
<code>package.json</code>	Root-level npm configuration (not used for building; may be a workspace meta file).
<code>deploy.sh</code>	Shell script for deployment automation.
<code>start-dev.bat</code> / <code>start-dev.ps1</code>	Windows convenience scripts to launch both servers for local development.
<code>AInotes.txt</code>	Critical developer notes. Written by the primary developer to guide AI assistants. Contains workflow principles, architecture boundaries, recurring bug patterns, and session summaries. Any AI working on this project should read this file first.
<code>EquationalReasoningPlan.txt</code>	Step-by-step plan that was followed to build the equational reasoning module. Documents architectural decisions made and tracks completion status.
<code>EquationalReasoningStatus.txt</code>	Current status summary of the equational reasoning feature.
<code>LIST_INDUCTION_PROGRESS.md</code>	Tracks the in-progress work to support list induction (as distinct from integer induction).
<code>eqrnPlan.txt</code>	Earlier planning notes for equational reasoning.
<code>TESTING.md</code>	Catalog of every automated test file and how each suite is run (Django, Jest, expression-tree scripts, live-server scripts).
<code>TESTING_MANUAL_QA.md</code>	Manual browser QA checklist for equational reasoning (not the automated suite).
<code>test_equational_api.py</code>	Standalone integration test script for the equational reasoning API (runs outside Django test runner).
<code>trial.ps1</code> / <code>trial-automation.js</code>	Scripts for automated testing of proof workflows.

File	Purpose
<code>make_proofbuddy_induction_ppt.py</code>	Utility to generate a PowerPoint presentation describing the induction proof system. (Research/presentation artifact.)
<code>hooks.json</code>	Git hooks configuration.

client/ — React Frontend

The entire browser-side application. Built with React 18, bundled with Create React App / Babel, styled with Bootstrap 5 + SCSS.

client/src/

Entry points:

- `index.js` — Mounts the React application into the DOM.
- `App.js` — Wraps the app in `AuthProvider` context and renders `AppRoutes`.
- `App.css` / `index.scss` — Global styles.

client/src/routes/ — React Router route definitions

File	Purpose
<code>index.js</code>	Root router using <code>HashRouter</code> . Composes Auth, Verification, and Proof sub-route groups.
<code>AuthRoutes.js</code>	Routes for <code>/login</code> , <code>/signup</code> , <code>/forgot-password</code> , etc.
<code>VerificationRoutes.js</code>	Routes for email verification and account activation flows.
<code>ProofRoutes.js</code>	Routes for all proof-mode pages (equational, induction, natural deduction). Protected — requires authentication.

client/src/pages/ — Top-level page components (one per URL path)

File	Purpose
<code>Home.js</code>	Landing page. Shows available proof types. Routes the user to the selected editor.
<code>EquationalReasoningNew.js</code>	Primary equational reasoning editor (~400 lines). The clean, modernized version of the equational reasoning UI. Uses the <code>equational_reasoning_api</code> backend.
<code>EquationalReasoning.js</code>	Older equational reasoning page (preserved as fallback/reference).
<code>ERRacket.js</code> / <code>EquationalReasoningRacket.js</code>	Legacy equational editors backed by the old <code>racket_api</code> . Still functional. Do not modify without caution.

File	Purpose
InductionRacket.js	Mathematical induction editor (~2400 lines). Handles both integer and list induction. Includes base/leap case switching, IH display, and full rule application UI.
NaturalDeductionPropositionalLogic.js	Legacy TFL/propositional logic page.
NaturalDeductionFirstOrderLogic.js	Legacy FOL page.
Login.js / SignUp.js / ForgotPassword.js	Authentication pages.
PageNotFound.js	404 catch-all page.

[client/src/components/](#) — Reusable React components

File	Purpose
PersistentPad.js	Core proof-line interaction component. Renders a single proof line as syntax-highlighted text with clickable nodes. Supports arrow-key navigation up/down/left/right through the expression tree (using jsonTree) and stores node selections in sessionStorage . Exposes ref methods for the parent page to read the currently selected node ID and rule text.
Definitions.jsx	UI for managing user-defined functions (UDFs). Allows creating, editing, enabling/disabling, and deleting definitions. Default (built-in) UDFs displayed read-only. Persists to sessionStorage .
Substitution.jsx	Modal dialog for applying eval rules that require a substitution expression. User enters the expression to match and the rule to apply.
ClickableRowNumber.js	A row-number widget that can be clicked to bind a rule application to a specific proof line.
RacketInput.jsx	Input field with parenthesis-balanced highlighting for Racket expressions.
ProofComplete.jsx	Overlay with confetti animation triggered when checkComplete returns true.
RuleSet.js / OffcanvasRuleSet.js	Reference cards listing all available proof rules and their syntax. Displayed as a sidebar or offcanvas panel.
Header.js / Footer.js	Site-wide layout chrome.

[client/src/services/](#) — API call wrappers (Axios-based)

File	Purpose
equationalService.js	Wrappers for all /api/v1/equational/... endpoints.
inductionService.js	Wrappers for all /api/v1/induction/... endpoints.
authService.js	Wrappers for login, signup, logout, password reset.

File	Purpose
<code>proofsService.js</code>	Wrappers for saving, loading, and deleting proofs.

`client/src/context/` — React Context providers

- `AuthProvider`: Holds the current user's auth token and user info in React context. Consumed throughout the component tree to determine login state and user identity.

`client/src/hooks/` — Custom React hooks

- `useRacketRuleFields`: Manages the array of proof-line field state (racket expression, rule, start position, selected node, visibility flags, errors). Used by both `EquationalReasoningNew.js` and `InductionRacket.js` to keep proof-line UI state DRY.

`client/src/config/` — Configuration

- Contains the base URL for the backend API, read from the `REACT_APP_BACKEND_API_BASE_URL` environment variable at build time.

`client/src/hoc/` — Higher-order components

- Authentication guards; wrap proof routes to redirect unauthenticated users to `/login`.

`client/src/utils/` — Utility functions

- Shared helper functions used across pages and components.

`client/src/scss/` — SCSS stylesheets

- Component-level styles, custom Bootstrap overrides, color variables.

`django_server/` — Python/Django Backend

`django_server/django_server/` — Django project package

File	Purpose
<code>settings.py</code>	Global configuration: database (MySQL), cache (db-based, 30-min timeout), installed apps, CORS allowed origins, DRF auth classes, email (SMTP).
<code>urls.py</code>	Root URL router. Mounts all Django app URL configurations under <code>/api/v1/</code> .
<code>asgi.py</code> / <code>wsgi.py</code>	ASGI/WSGI entry points for deployment.

`django_server/expression_tree/` — Proof Engine (core library)

This is the most intellectually significant directory in the project. It is a pure-Python library that implements all proof logic. It has no Django dependencies — it could be extracted and used independently.

File	Purpose
Parser.py	Tokenizer and AST builder. <code>preProcess()</code> validates and tokenizes a Racket-like expression string. <code>buildTree()</code> recursively builds a <code>Node</code> tree from the token list. <code>makeBasicAst()</code> is the main public entry point combining both steps.
ERCommon.py	Core data structures. Defines <code>Node</code> (AST node), <code>RacType</code> (type representation), the <code>Type</code> enum, and tree utility functions including <code>findNode()</code> (locate a node by start position) and <code>makeJson()</code> (convert a tree to the JSON format sent to the frontend).
ERObj.py	Built-in operator definitions. Defines <code>ERObj</code> descriptors and a library of pre-built primitive operations: arithmetic (<code>+</code> , <code>-</code> , <code>*</code> , <code>expt</code> , <code>quotient</code> , <code>remainder</code>), comparisons (<code>=</code> , <code>></code> , <code><</code>), logic (<code>and</code> , <code>or</code> , <code>not</code> , <code>xor</code> , <code>implies</code>), list operations (<code>cons</code> , <code>first</code> , <code>rest</code>), type predicates (<code>null?</code> , <code>zero?</code> , <code>integer?</code> , <code>list?</code>), and <code>if</code> .
expressionDefinition.py	Type labeler. <code>labelTree()</code> walks a freshly-parsed Node tree and assigns types to all nodes by looking up built-ins and UDFs in lookup tables. This is a first pass before full type checking.
Decorator.py	Type checker / propagator. Walks the type-labeled tree and propagates type information through the tree, detecting type mismatches and recording errors.
ERGenerics.py	Generic variable system. Defines <code>GenericInt</code> , <code>GenericList</code> , <code>GenericBool</code> , <code>GenericAny</code> . Generics represent unspecified variables with typed constraints (e.g., "a non-negative integer"). Used in proof contexts where variables haven't been given concrete values.
ERRuleset.py	All proof rules. Defines an abstract <code>Rule</code> class and all concrete rule types: <code>BuiltIn</code> , <code>Math</code> (with SymPy), <code>Axiom</code> , <code>UDF</code> , <code>IH</code> , <code>If</code> . Each rule implements <code>isApplicable()</code> (can this rule be applied here?) and <code>insertSubstitution()</code> (what is the result?). The <code>isMatch()</code> helper checks structural tree equality.
ERProofEngine.py	Proof session management. Defines <code>ERProofLine</code> (one step), <code>ERProof</code> (one side of a proof), <code>TwoSidedProof</code> (equational reasoning proof), and <code>ProofComponent</code> (shared base class). <code>addProofLine()</code> is the main entry point for rule application. <code>checkComplete()</code> verifies LHS and RHS convergence.
IndProofs.py	Induction proof structure. Defines <code>IndProof</code> , which wraps two <code>TwoSidedProof</code> objects (one for the base case, one for the leap step) and adds metadata for the induction variable, anchor value, leap variable, and inductive hypothesis.
Labeler.py	Helper for expression labeling.
default_udfs.py	Pre-built user-defined functions. Provides <code>length</code> , <code>append</code> , and <code>countTruthTableRows</code> as built-in starter definitions available to all

File	Purpose
	users. These are not stored in the database (they have negative <code>id</code> values).
<code>sluff.py</code>	Utility functions for tree manipulation.
<code>runTests.py</code> / <code>testAdvMath.py</code> / <code>testApplyRule.py</code>	Unit test files for the expression engine.

`django_server/equational_reasoning_api/` — Equational Reasoning Django App

File	Purpose
<code>models.py</code>	<code>EquationalProof</code> (proof metadata) and <code>EquationalProofLine</code> (individual proof steps). See database schema section below.
<code>views.py</code>	All equational reasoning API endpoints. Six views: <code>set_current_proof</code> , <code>apply_rule</code> , <code>substitution</code> , <code>delete_line</code> , <code>check_completion</code> , <code>get_proof_lines</code> . Each view reconstructs the proof from cache/DB, calls the proof engine, saves results, and returns JSON.
<code>serializers.py</code>	DRF serializers for <code>EquationalProof</code> . Validates that LHS and RHS goals are non-empty and not identical.
<code>urls.py</code>	URL patterns for all equational endpoints, mounted at <code>/api/v1/equational/</code> .
<code>tests.py</code>	11 automated tests for models, serializers, and proof behavior.
<code>test_integration.py</code>	Additional integration tests.

`django_server/induction_api/` — Induction Django App

File	Purpose
<code>models.py</code>	<code>InductionProof</code> and <code>InductionProofLine</code> . Similar to equational models but includes <code>case</code> (base/leap), <code>proof_type</code> (int/list), IH fields, and induction variable metadata.
<code>views.py</code>	Endpoints: <code>start_induction_proof</code> , <code>apply_rule</code> , <code>substitution</code> , <code>delete_line</code> , <code>check_goal</code> , <code>get_proof_lines</code> , and others. Same overall pattern as equational but includes <code>case</code> context throughout.
<code>urls.py</code>	URL patterns mounted at <code>/api/v1/induction/</code> .
<code>README.md</code>	Internal documentation for the induction API.

`django_server/accounts/` — User Authentication App

File	Purpose
<code>models.py</code>	<code>Account</code> (custom user model extending <code>AbstractBaseUser</code>): email, username, first/last name, <code>is_instructor</code> flag, <code>is_active</code> . Also <code>ActivateAccount</code> for email verification tokens.
<code>views.py</code>	Login, signup, token creation, password reset, email verification, username lookup.
<code>urls.py</code>	Auth URL patterns mounted at <code>/api/v1/auth/</code> .
<code>serializers.py</code>	Serializers for account creation and retrieval.

`django_server/proofs/` — Legacy Proof Storage App

File	Purpose
<code>models.py</code>	<code>Definition</code> , <code>Generic</code> , <code>Proof</code> , <code>ProofLine</code> . Original data models from before the equational/induction split. <code>Proof</code> supports multiple "templates" (ER, MP, MT, DS, ADD).
<code>views.py</code>	CRUD for definitions, generics, and legacy proofs.
<code>urls.py</code>	URL patterns mounted at <code>/api/v1/proofs</code> .

`django_server/racket_api/` — Legacy Proof Validation App

Provides the backend for the legacy `ERRacket.js` frontend. Contains proof validation logic that predates the current `expression_tree` architecture. Should not be modified.

`django_server/assignments/` — Assignment Management App

File	Purpose
<code>models.py</code>	<code>Course</code> (course section), <code>Assignment</code> (problem set with due date), <code>Course</code> has an instructor and a set of student <code>Account</code> references.
<code>views.py</code>	Instructor CRUD for terms and assignments; student submission endpoints.

`database/` — Database Setup Scripts

File	Purpose
<code>docker-compose.yml</code>	Docker Compose configuration for running just the MySQL database in a container (useful for development without a full local MySQL install).
<code>setup.sql</code>	Initial SQL script to create the database and user.

`nginx/` — Reverse Proxy Configuration

File	Purpose
<code>nginx.conf</code>	Nginx configuration. Defines an <code>upstream</code> pointing to <code>django_server:8000</code> . Routes all requests under <code>/</code> to Django and serves <code>/static/</code> files from a shared Docker volume.

documentation/ — Project Documentation

Directory	Purpose
<code>docker/</code>	Docker setup guides.
<code>readme_resources/</code>	Detailed local installation guides: first-time setup, <code>.env</code> file creation, running the application, API reference.
<code>use_cases/</code>	Use case diagrams or descriptions of user workflows.
<code>user_documentation/</code>	End-user and developer documentation including the API reference.

django_server/ Root-Level Files

File	Purpose
<code>manage.py</code>	Django management command entry point.
<code>requirements.txt</code>	Python package dependencies.
<code>Dockerfile</code>	Docker image definition for the Django server.
<code>entrypoint.sh</code>	Docker container startup script (runs migrations, collectstatic, starts Gunicorn).
<code>gunicorn.conf.py</code>	Gunicorn WSGI server configuration (workers, bind address).
<code>test_manual_persistence.py</code>	Developer script for manually testing proof persistence behavior.